SPARTAN6-DSP48A1

Prepared by:

Arwa Kantoush

TABLE OF CONTENTS

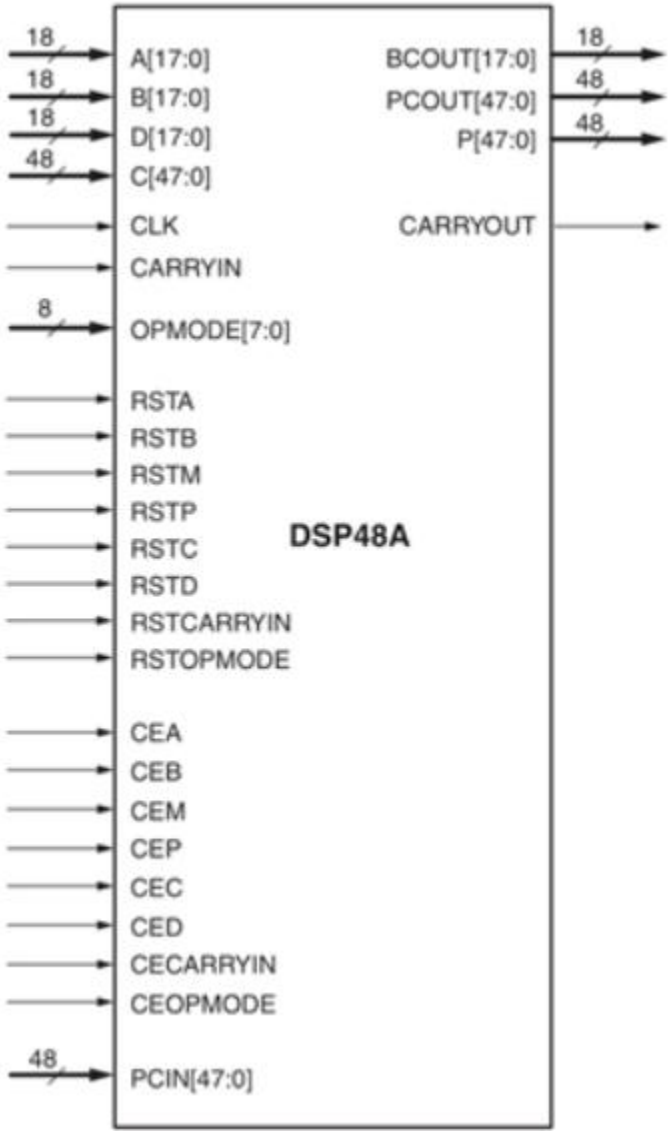
8. IMPLEMENTATION FLOW (PLACE & ROUTE)

- 8.1 Device Placement
- 8.2 Timing Summary
- 8.3 Utilization Report
- 8.4 Power Consumption
- 8.5 messages

9. LINTING

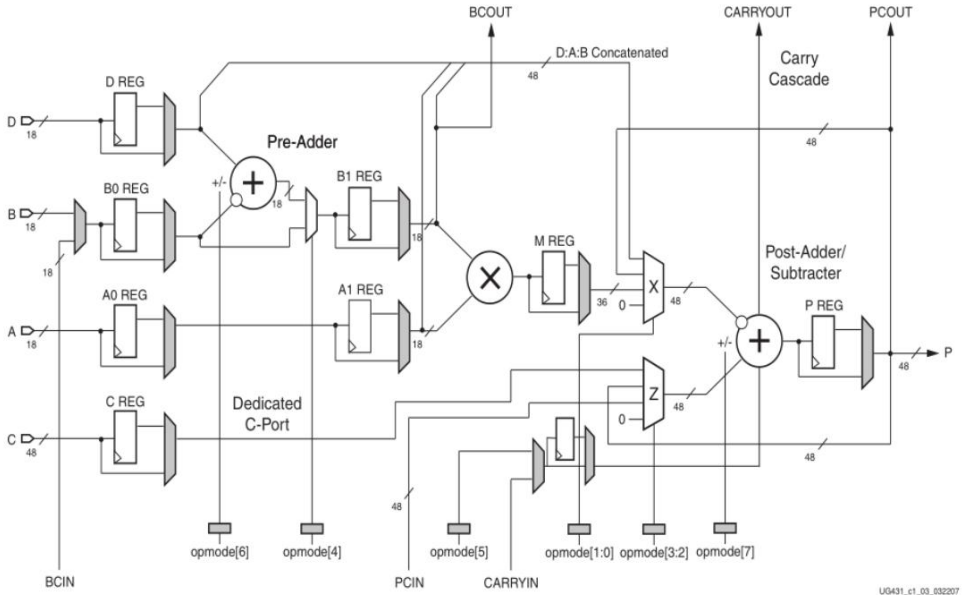
1. INTRODUCTION & OVERVIEW

1.1 Project Overview & Objectives



This project presents the design, modeling, functional verification, and FPGA implementation of the **Spartan-6 DSP48A1 slice** using **Verilog HDL**. The DSP48A1 block is a specialized arithmetic unit tailored for digital signal processing (DSP) applications, matrix multiplication, and wide multi-precision arithmetic. The entire implementation follows the complete FPGA design flow utilizing **Mentor Graphics QuestaSim** for simulation and **AMD Xilinx Vivado** for elaboration, synthesis, implementation, and linting targeting the Artix-7 FPGA (xc7a200tffg1156-3).

1.2 Spartan-6 DSP48A1 Architecture Summary



The DSP48A1 slice consists of four core arithmetic and routing stages:

- **18-bit Pre-Adder / Subtractor:** Controlled via OPMODE[6] and OPMODE[4], allowing dynamic pre-addition/subtraction $(D \pm B)$ before multiplication to optimize symmetric filter operations.
- 18×18**-bit Multiplier:** Computes a full 36-bit two's complement product ($A \times B$ or $A \times (D \pm B)$).
- **48-bit Post-Adder / Subtractor / Accumulator:** Dynamically computes wide additions, subtractions, and accumulations $\left(Z \pm (X + \text{CIN})\right)$ through multiplexers X and Z.
- **Cascading Routing Buses:** Dedicated 18-bit (BCIN/BCOUT) and 48-bit (PCIN/PCOUT) paths allow chaining multiple slices together without general FPGA interconnect overhead.

1.3 Operational Features & Pipeline Stages

- **Dynamic Operation Selection:** An 8-bit dynamic control bus (OPMODE[7:0]) controls input selection for multiplexers X and Z, pre-adder bypass/operation, carry-in routing, and post-adder subtraction on a per-cycle basis.
- **Configurable Pipeline Registers:** Every path ($A0, A1, B0, B1, C, D, M, P$,CYI,CYO,OPMODE) includes parameter-controlled pipeline registers to maximize throughput and achieve high clock frequencies.
- **Selectable Reset Mode:** Supports either synchronous (SYNC) or asynchronous (ASYNC) active-high reset behavior across all internal registers.

2. RTL Design & Modeling

2.1 Parameterized Register Module (DSP_REG.v)

```
module DSP_REG #(
    parameter WIDTH = 18,
    parameter REG = 1,
    parameter  RSTTYPE = "SYNC"
) (
    input CLK,
    input rst,
    input ce,
    input [WIDTH-1:0]in,
    output reg [WIDTH-1:0]out
);


generate
    if (REG && RSTTYPE == "SYNC") begin : GEN_SYNC
        always @(posedge CLK) begin
            if (rst) begin
                out <= 0;
            end
            else if (ce) begin
                out <= in;
            end
        end
    end
    else if (REG && RSTTYPE == "ASYNC") begin : GEN_ASYNC
        always @(posedge CLK or posedge rst) begin
            if (rst) begin
                out <= 0;
            end
            else if (ce) begin
                out <= in;
            end
        end
    end
    else begin : GEN_BYPASS
        always @(*) begin
            out = in;
        end
    end
endgenerate
endmodule //DSP_reg
```

2.2 Top-Level Slice Module (DSP.v)

```
module DSP #(
    parameter WIDTH_18 = 18,
    parameter WIDTH_48 = 48,
    parameter A0REG = 0,
    parameter A1REG = 1,
    parameter B0REG = 0,
    parameter B1REG = 1,
    parameter CREG = 1,
    parameter DREG = 1,
    parameter MREG = 1,
    parameter PREG = 1,
    parameter CARRYINREG = 1,
    parameter CARRYOUTREG = 1,
    parameter OPMODEREG = 1,
    parameter CARRYINSEL = "OPMODE5",
    parameter B_INPUT = "DIRECT",
    parameter  RSTTYPE = "SYNC"
) (
    input CLK,
    input [7:0]OPMODE,
    input CEA,
    input CEB,
    input CEC,
    input CECARRYIN,
    input CED,
    input CEM,
    input CEOPMODE,
    input CEP,
    input RSTA,
    input RSTB,
    input RSTC,
    input RSTCARRYIN,
    input RSTD,
    input RSTM,
    input RSTOPMODE,
    input RSTP,
    input [WIDTH_18-1:0]A,
    input [WIDTH_18-1:0]B,
    input [WIDTH_18-1:0]BCIN,
    input [WIDTH_48-1:0]C,
    input [WIDTH_18-1:0]D,
    input CARRYIN,
    input [WIDTH_48-1:0]PCIN,
```

```
    output [35:0]M,
    output [WIDTH_48-1:0]P,
    output CARRYOUT,
    output CARRYOUTF,
    output [WIDTH_18-1:0]BCOUT,
    output [WIDTH_48-1:0]PCOUT
);
wire [WIDTH_18-1:0]A0_REG,A1_REG,B_SEL,B0_REG,B1_REG,D_REG,SUM1,SUM1_SEL;
wire [35:0]MUL,M_REG;
wire [WIDTH_48-1:0]C_REG;
wire [WIDTH_48:0]P_REG;
reg [WIDTH_48-1:0]SEL_X,SEL_Z;
wire CYI_SEL,CYI;
wire [7:0]OPMODE_REG;

DSP_REG #(.WIDTH(8),.REG(OPMODEREG),.RSTTYPE(RSTTYPE)) dutOPMODE
(.CLK(CLK),.rst(RSTOPMODE),.ce(CEOPMODE),.in(OPMODE),.out(OPMODE_REG));
DSP_REG #(.WIDTH(WIDTH_18),.REG(DREG),.RSTTYPE(RSTTYPE)) dutD
(.CLK(CLK),.rst(RSTD),.ce(CED),.in(D),.out(D_REG));

generate
    if (B_INPUT == "DIRECT") begin : GEN_B_DIRECT
        assign B_SEL = B;
    end
    else if (B_INPUT == "CASCADE") begin : GEN_B_CASCADE
        assign B_SEL = BCIN;
    end
    else begin : GEN_B_ZERO
        assign B_SEL = 18'b0;
    end
endgenerate

DSP_REG #(.WIDTH(WIDTH_18),.REG(B0REG),.RSTTYPE(RSTTYPE)) dutB0
(.CLK(CLK),.rst(RSTB),.ce(CEB),.in(B_SEL),.out(B0_REG));
assign SUM1 = (OPMODE_REG[6] == 1'b0)? (D_REG + B0_REG) : (D_REG -
B0_REG);
assign SUM1_SEL = (OPMODE_REG[4] == 1'b0)? B0_REG : SUM1;
DSP_REG #(.WIDTH(WIDTH_18),.REG(B1REG),.RSTTYPE(RSTTYPE)) dutB1
(.CLK(CLK),.rst(RSTB),.ce(CEB),.in(SUM1_SEL),.out(B1_REG));
DSP_REG #(.WIDTH(WIDTH_18),.REG(A0REG),.RSTTYPE(RSTTYPE)) dutA0
(.CLK(CLK),.rst(RSTA),.ce(CEA),.in(A),.out(A0_REG));
DSP_REG #(.WIDTH(WIDTH_18),.REG(A1REG),.RSTTYPE(RSTTYPE)) dutA1
(.CLK(CLK),.rst(RSTA),.ce(CEA),.in(A0_REG),.out(A1_REG));
assign MUL = B1_REG * A1_REG;
DSP_REG #(.WIDTH(36),.REG(MREG),.RSTTYPE(RSTTYPE)) dutM
(.CLK(CLK),.rst(RSTM),.ce(CEM),.in(MUL),.out(M_REG));
```

```
always @(*) begin
    case (OPMODE_REG[1:0])
        2'b00: SEL_X = 0;
        2'b01: SEL_X = {12'b0,M_REG};
        2'b10: SEL_X = P;
        2'b11: SEL_X = {D_REG[11:0],A1_REG[17:0],B1_REG[17:0]};
        default : SEL_X = 48'b0;
    endcase
end
DSP_REG #(.WIDTH(WIDTH_48),.REG(CREG),.RSTTYPE(RSTTYPE)) dutC
(.CLK(CLK),.rst(RSTC),.ce(CEC),.in(C),.out(C_REG));
always @(*) begin
    case (OPMODE_REG[3:2])
        2'b00: SEL_Z = 0;
        2'b01: SEL_Z = PCIN;
        2'b10: SEL_Z = P;
        2'b11: SEL_Z = C_REG;
        default : SEL_Z = 48'b0;
    endcase
end
generate
    if (CARRYINSEL == "OPMODE5") begin : GEN_CYI_OPMODE5
        assign CYI_SEL = OPMODE_REG[5];
    end
    else if (CARRYINSEL == "CARRYIN") begin : GEN_CYI_CARRYIN
        assign CYI_SEL = CARRYIN;
    end
    else begin : GEN_CYI_ZERO
        assign CYI_SEL = 1'b0;
    end
endgenerate
DSP_REG #(.WIDTH(1),.REG(CARRYINREG),.RSTTYPE(RSTTYPE)) dutCYI
(.CLK(CLK),.rst(RSTCARRYIN),.ce(CECARRYIN),.in(CYI_SEL),.out(CYI));
assign P_REG = (OPMODE_REG[7] == 1'b0)? (SEL_Z + SEL_X + CYI) : (SEL_Z -
(SEL_X + CYI));
DSP_REG #(.WIDTH(WIDTH_48),.REG(PREG),.RSTTYPE(RSTTYPE)) dutP
(.CLK(CLK),.rst(RSTP),.ce(CEP),.in(P_REG[WIDTH_48-1:0]),.out(P));
DSP_REG #(.WIDTH(1),.REG(CARRYOUTREG),.RSTTYPE(RSTTYPE)) dutCYO
(.CLK(CLK),.rst(RSTCARRYIN),.ce(CECARRYIN),.in(P_REG[WIDTH_48]),.out(CARRY
OUT));
assign CARRYOUTF = CARRYOUT;
assign M = M_REG;
assign BCOUT = B1_REG;
assign PCOUT = P;
endmodule //DSP
```

3. FUNCTIONAL VERIFICATION & TESTBENCH

```
module DSP_tb ();
parameter WIDTH_18 = 18;
parameter WIDTH_48 = 48;
parameter A0REG = 0;
parameter A1REG = 1;
parameter B0REG = 0;
parameter B1REG = 1;
parameter CREG = 1;
parameter DREG = 1;
parameter MREG = 1;
parameter PREG = 1;
parameter CARRYINREG = 1;
parameter CARRYOUTREG = 1;
parameter OPMODEREG = 1;
parameter CARRYINSEL = "OPMODE5";
parameter B_INPUT = "DIRECT";
parameter RSTTYPE = "SYNC";

reg CLK;
reg [7:0]OPMODE;
reg CEA;
reg CEB;
reg CEC;
reg CECARRYIN;
reg CED;
reg CEM;
reg CEOPMODE;
reg CEP;
reg RSTA;
reg RSTB;
reg RSTC;
reg RSTCARRYIN;
reg RSTD;
reg RSTM;
reg RSTOPMODE;
reg RSTP;
reg [WIDTH_18-1:0]A;
reg [WIDTH_18-1:0]B;
reg [WIDTH_18-1:0]BCIN;
reg [WIDTH_48-1:0]C;
reg [WIDTH_18-1:0]D;
reg CARRYIN;
```

```
reg [WIDTH_48-1:0]PCIN;
wire [35:0]M;
wire [WIDTH_48-1:0]P;
wire CARRYOUT;
wire CARRYOUTF;
wire [WIDTH_18-1:0]BCOUT;
wire [WIDTH_48-1:0]PCOUT;
reg [35:0]M_exp;
reg [WIDTH_48-1:0]P_exp;
reg CARRYOUT_exp;
reg CARRYOUTF_exp;
reg [WIDTH_18-1:0]BCOUT_exp;
reg [WIDTH_48-1:0]PCOUT_exp;

DSP #(
    .WIDTH_18(WIDTH_18),
    .WIDTH_48(WIDTH_48),
    .A0REG(A0REG),
    .A1REG(A1REG),
    .B0REG(B0REG),
    .B1REG(B1REG),
    .CREG(CREG),
    .DREG(DREG),
    .MREG(MREG),
    .PREG(PREG),
    .CARRYINREG(CARRYINREG),
    .CARRYOUTREG(CARRYOUTREG),
    .OPMODEREG(OPMODEREG),
    .CARRYINSEL(CARRYINSEL),
    .B_INPUT(B_INPUT),
    .RSTTYPE(RSTTYPE)
) dut (
    .CLK(CLK),
    .OPMODE(OPMODE),
    .CEA(CEA),
    .CEB(CEB),
    .CEC(CEC),
    .CECARRYIN(CECARRYIN),
    .CED(CED),
    .CEM(CEM),
    .CEOPMODE(CEOPMODE),
    .CEP(CEP),
    .RSTA(RSTA),
    .RSTB(RSTB),
    .RSTC(RSTC),
    .RSTCARRYIN(RSTCARRYIN),
```

```
    .RSTD(RSTD),
    .RSTM(RSTM),
    .RSTOPMODE(RSTOPMODE),
    .RSTP(RSTP),
    .A(A),
    .B(B),
    .BCIN(BCIN),
    .C(C),
    .D(D),
    .CARRYIN(CARRYIN),
    .PCIN(PCIN),
    .M(M),
    .P(P),
    .CARRYOUT(CARRYOUT),
    .CARRYOUTF(CARRYOUTF),
    .BCOUT(BCOUT),
    .PCOUT(PCOUT)
);

initial begin
    CLK = 1;
    forever begin
        #1 CLK = ~CLK;
    end
end

initial begin
    //1
    RSTA = 1; RSTB = 1; RSTC = 1; RSTCARRYIN = 1; RSTD = 1; RSTM = 1;
RSTOPMODE = 1; RSTP = 1;
    CEA = $random; CEB = $random; CEC = $random; CECARRYIN = $random; CED
= $random; CEM = $random; CEOPMODE = $random; CEP = $random;
    A = $random; B = $random; BCIN = $random; C = $random; D = $random;
OPMODE = $random; CARRYIN = $random; PCIN = $random;
    M_exp = 0; P_exp = 0; CARRYOUT_exp = 0; CARRYOUTF_exp = 0; BCOUT_exp =
0; PCOUT_exp = 0;
    @(negedge CLK);
    if (M!==M_exp || P!==P_exp || CARRYOUT!==CARRYOUT_exp ||
CARRYOUTF!==CARRYOUTF_exp || BCOUT!==BCOUT_exp || PCOUT!==PCOUT_exp) begin
        $display("ERROR!");
        $stop;
    end
    //2
    RSTA = 0; RSTB = 0; RSTC = 0; RSTCARRYIN = 0; RSTD = 0; RSTM = 0;
RSTOPMODE = 0; RSTP = 0;
```

```
    CEA = 1; CEB = 1; CEC = 1; CECARRYIN = 1; CED = 1; CEM = 1; CEOPMODE =
1; CEP = 1;
    A = 20; B = 10; BCIN = $random; C = 350; D = 25; OPMODE = 8'b11011101;
CARRYIN = $random; PCIN = $random;
    M_exp = 'h12c; P_exp = 'h32; CARRYOUT_exp = 0; CARRYOUTF_exp = 0;
BCOUT_exp = 'hf; PCOUT_exp = 'h32;
    repeat(4) @(negedge CLK);
    if (M!==M_exp || P!==P_exp || CARRYOUT!==CARRYOUT_exp ||
CARRYOUTF!==CARRYOUTF_exp || BCOUT!==BCOUT_exp || PCOUT!==PCOUT_exp) begin
        $display("ERROR!");
        $stop;
    end
    //3
    OPMODE = 8'b00010000;
    M_exp = 'h2bc; P_exp = 'h0; CARRYOUT_exp = 0; CARRYOUTF_exp = 0;
BCOUT_exp = 'h23; PCOUT_exp = 'h0;
    repeat(3) @(negedge CLK);
    if (M!==M_exp || P!==P_exp || CARRYOUT!==CARRYOUT_exp ||
CARRYOUTF!==CARRYOUTF_exp || BCOUT!==BCOUT_exp || PCOUT!==PCOUT_exp) begin
        $display("ERROR!");
        $stop;
    end
    //4
    OPMODE = 8'b00001010;
    M_exp = 'hc8; P_exp = 'h0; CARRYOUT_exp = 0; CARRYOUTF_exp = 0;
BCOUT_exp = 'ha; PCOUT_exp = 'h0;
    repeat(3) @(negedge CLK);
    if (M!==M_exp || P!==P_exp || CARRYOUT!==CARRYOUT_exp ||
CARRYOUTF!==CARRYOUTF_exp || BCOUT!==BCOUT_exp || PCOUT!==PCOUT_exp) begin
        $display("ERROR!");
        $stop;
    end
    //5
    A = 5; B = 6; C = 350; D = 25; PCIN = 3000;
    OPMODE = 8'b10100111;
    M_exp = 'h1e; P_exp = 'hfe6fffec0bb1; CARRYOUT_exp = 1; CARRYOUTF_exp
= 1; BCOUT_exp = 'h6; PCOUT_exp = 'hfe6fffec0bb1;
    repeat(3) @(negedge CLK);
    if (M!==M_exp || P!==P_exp || CARRYOUT!==CARRYOUT_exp ||
CARRYOUTF!==CARRYOUTF_exp || BCOUT!==BCOUT_exp || PCOUT!==PCOUT_exp) begin
        $display("ERROR!");
        $stop;
    end
    $stop;
end
```

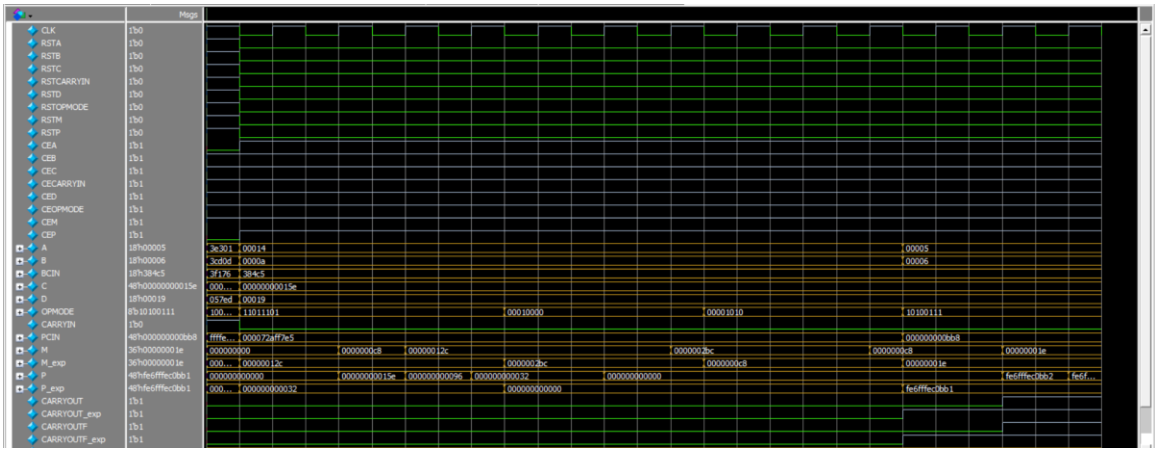
```
initial begin
    $monitor("M=%h,M_exp=%h,P=%h,P_exp=%h,PCOUT=%h,PCOUT_exp=%h,CARRYOUT=%
h,CARRYOUT_exp=%h,CARRYOUTF=%h,CARRYOUTF_exp=%h,BCOUT=%h,BCOUT_exp=%h",M,M
_exp,P,P_exp,PCOUT,PCOUT_exp,CARRYOUT,CARRYOUT_exp,CARRYOUTF,CARRYOUTF_exp
,BCOUT,BCOUT_exp);
end
endmodule //DSP_tb
```

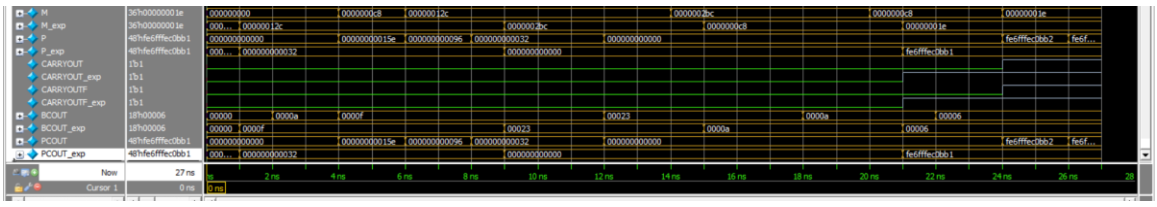
4. SIMULATION & AUTOMATION

4.1 QuestaSim Simulation Script (run.do)

```
vlib work
vlog DSP_REG.v DSP.v DSP_tb.v
vsim -voptargs=+acc work.DSP_tb
add wave *
run -all
#quit -sim
```

4.2 Simulation Waveforms & Output Analysis





5. DESIGN CONSTRAINTS

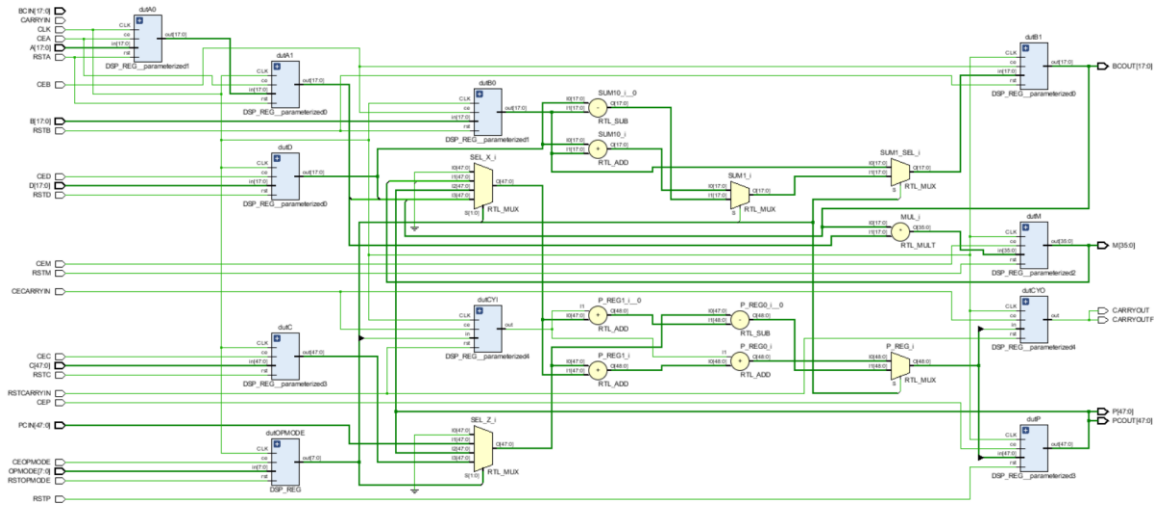
```
## Clock signal
set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports
CLK]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5}
[get_ports CLK]

## Configuration options, can be used for all designs
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]

## SPI configuration mode options for QSPI boot, can be used for all
designs
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
```

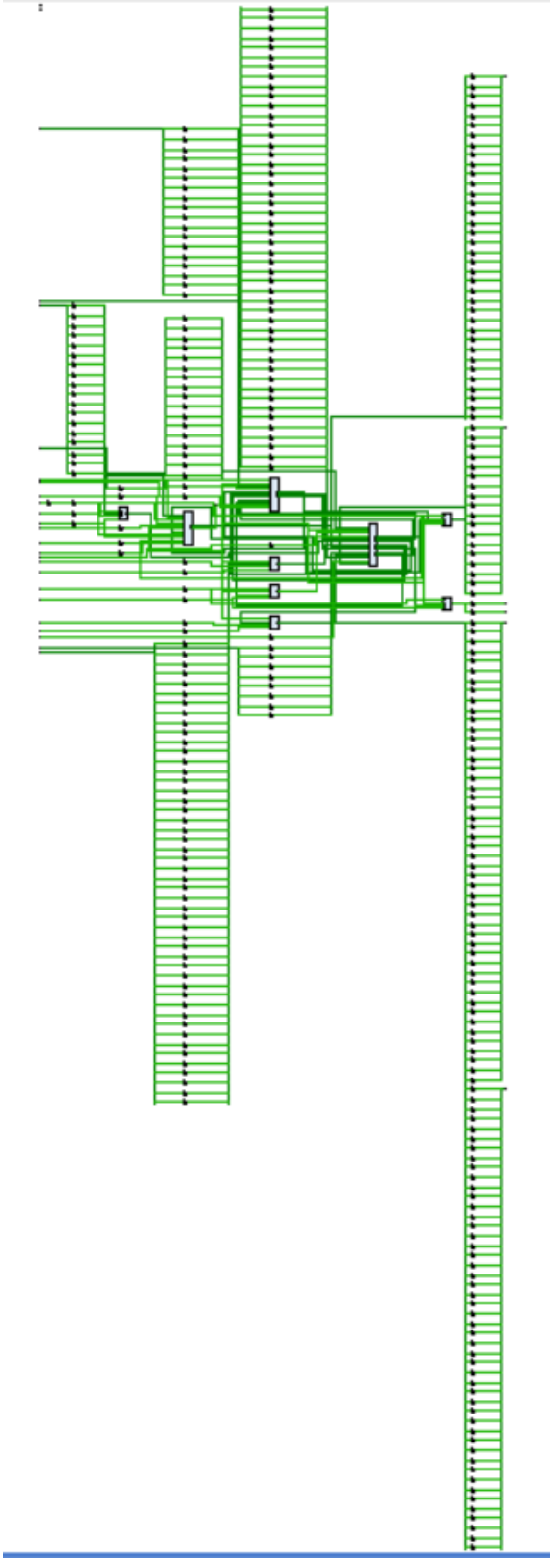
6. Elaboration Flow

6.1 Elaborated Schematic



7. SYNTHESIS FLOW

7.1 Synthesized Schematic



7.2 Timing Summary

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	5.168 ns	Worst Hold Slack (WHS):	0.182 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	106	Total Number of Endpoints:	106	Total Number of Endpoints:	162

All user specified timing constraints are met.

7.3 Utilization Report

Name 1	Slice LUTs (134600)	Slice Registers (269200)	DSPs (740)	Bonded IOB (500)	BUFGCTRL (32)
N DSP	230	160	1	327	1
dutA1 (DSP_REG__pa...	0	18	0	0	0
dutB1 (DSP_REG__pa...	0	18	0	0	0
dutC (DSP_REG__par...	0	48	0	0	0
dutCYI (DSP_REG__p...	1	1	0	0	0
dutCYO (DSP_REG__...	0	1	0	0	0
dutD (DSP_REG__par...	0	18	0	0	0
dutM (DSP_REG__par...	0	0	1	0	0
dutOPMODE (DSP_RE...	228	8	0	0	0
dutP (DSP_REG__par...	0	48	0	0	0

7.4 Power Consumption

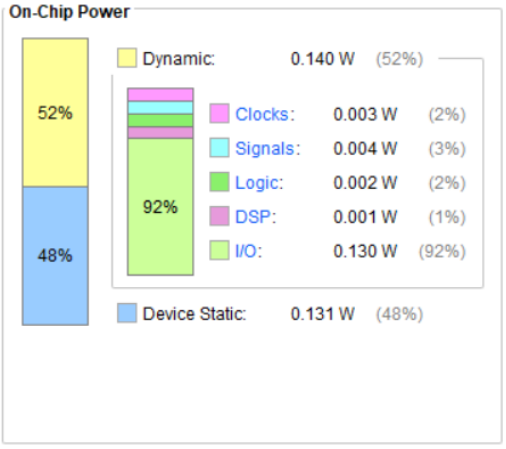
Summary

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	**0.271 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**25.4°C**
Thermal Margin:	74.6°C (50.8 W)
Effective ϑJA:	1.5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

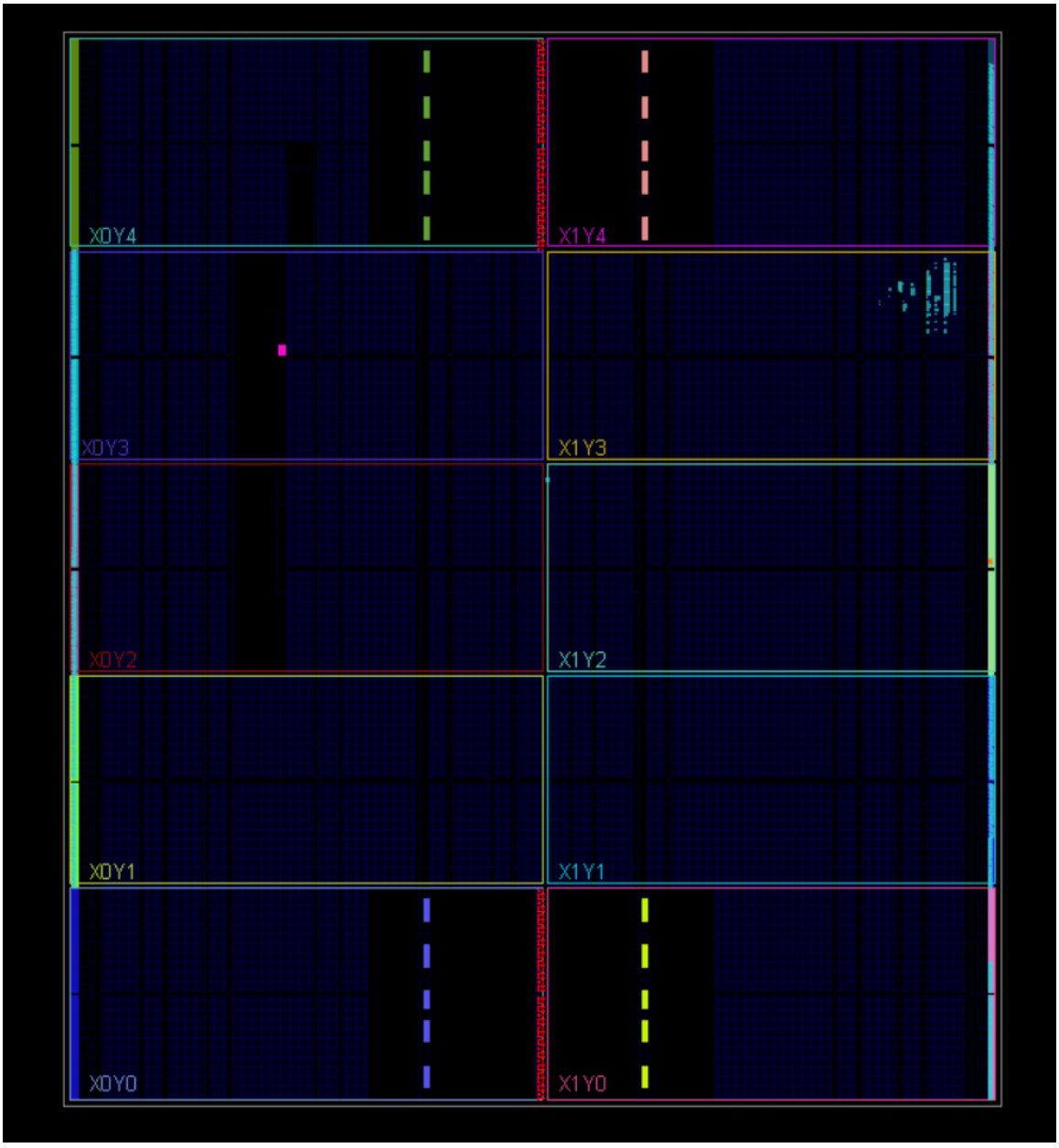
Launch Power Constraint Advisor to find and fix invalid switching activity

On-Chip Power



8. Implementation Flow (Place & Route)

8.1 Device Placement



8.2 Timing Summary

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	3.805 ns	Worst Hold Slack (WHS):	0.173 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	125	Total Number of Endpoints:	125	Total Number of Endpoints:	181

All user specified timing constraints are met.

8.3 Utilization Report

Name	Slice LUTs (133800)	Slice Registers (267600)	Slice (33450)	LUT as Logic (133800)	LUT Flip Flop Pairs (133800)	DSPs (740)	Bonded IOB (500)	BUFGCTRL (32)
N DSP	229	179	104	229	50	1	327	1
dutA1 (DSP_REG__pa...	0	18	8	0	0	0	0	0
dutB1 (DSP_REG__pa...	0	36	13	0	0	0	0	0
dutC (DSP_REG__par...	0	48	13	0	0	0	0	0
dutCYI (DSP_REG__p...	1	1	1	1	1	0	0	0
dutCYO (DSP_REG__...	0	2	2	0	0	0	0	0
dutD (DSP_REG__par...	0	18	9	0	0	0	0	0
dutM (DSP_REG__par...	0	0	0	0	0	1	0	0
dutOPMODE (DSP_RE...	228	8	72	228	0	0	0	0
dutP (DSP_REG__par...	0	48	12	0	0	0	0	0

8.4 Power Consumption

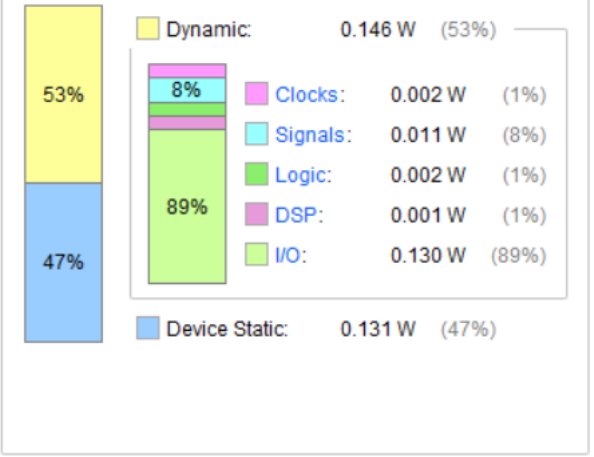
Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.277 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**25.4°C**
Thermal Margin:	74.6°C (50.8 W)
Effective ϑJA:	1.5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity

On-Chip Power



8.5 Messages

9. Linting